

# TD2 : Jointures externes, partitionnement et synthèse des requêtes — Corrigé enseignant d121886

Rosine Cicchetti - Lotfi Lakhal - Mickaël Martin Nevot



Cette œuvre de Rosine Cicchetti est mise à disposition sous licence Creative Commons  
Attribution - Utilisation non commerciale - Partage dans les mêmes conditions.

Document en ligne : [www.mickael-martin-nevot.com](http://www.mickael-martin-nevot.com)

---

## 1 Rappel : Prise en main d'un éditeur Oracle

Utilisez une des deux solutions ci-dessous.

### 1.1 Oracle Live SQL

Vous pouvez utiliser directement, sans installation ou configuration, l'interpréteur en ligne :  
<https://livesql.oracle.com>.

### 1.2 Oracle Cloud Free Tier

Mettez en place une solution d'hébergement en ligne d'un SGBD Oracle. Vous pouvez pour cela  
consulter le document *Vade-Mecum mise en place d'un hébergement Oracle Cloud Free Tier*.  
Ajouter ensuite une base de données nommée `zenetude-bd`.

## 2 Rappel : Généralités

Voici la convention de nommage proposé dans cet enseignement :

- **mots clefs** : en lettre capitales (*upper case*) ;
- **relation** : première lettre de chaque mot en capitale (*pascal case*) ;
- **attributs** : premier mot en minuscule et première lettre de chaque mot suivant en capitale (*camel case*) ;
- **domaine** : en lettre capitales (*upper case*) au format D\_XXX<sup>1</sup>.

Pour rappel, les valeurs saisies dans une base de données, comme les chaînes de caractères, sont sensibles à la casse et, par convention, sont saisies **en lettres capitales** (*upper case*), sans diacritique, dans le cadre de cet enseignement.

---

1. Les domaines identiques sont surlignés avec la même couleur.

### 3 Rappel : Base de données exemple

La base de données exemple, Voyage, utilisée dans les documents de travaux dirigés de cet enseignement, permet à un réseau d’agences de voyage de gérer les clients, les voyages ainsi que leurs options, la planification des voyages et les réservations des clients.

#### 3.1 Dictionnaire de données

La base de données Voyage a été élaborée à partir du dictionnaire des données suivant<sup>2</sup> :

**Table 1** – Dictionnaire des données de la base de données exemple

| Attribut  | Description                    | Domaine                       | Remarques         |
|-----------|--------------------------------|-------------------------------|-------------------|
| idV       | Identifiant d’un voyage        | D_IDV : NUMBER(6,0)           | Valeurs uniques   |
| villeArr  | Ville d’arrivée d’un voyage    | D_VILLE :<br>VARCHAR2(20)     |                   |
| paysArr   | Pays d’arrivée d’un voyage     | D_PAYS :<br>VARCHAR2(20)      |                   |
| villeDep  | Ville de départ d’un voyage    | D_VILLE :<br>VARCHAR2(20)     |                   |
| hotel     | Nom d’un hôtel                 | D_HOTEL :<br>VARCHAR2(20)     |                   |
| nbEtoiles | Nombre d’étoiles d’un hôtel    | D_NBETOILE :<br>NUMBER(1,0)   |                   |
| duree     | Nombre de jours d’un voyage    | D_DUREE :<br>NUMBER(2,0)      |                   |
| dateDep   | Date de départ                 | D_DATE : DATE                 |                   |
| tarif     | Prix unitaire du voyage        | D_TARIF :<br>NUMBER(6,2)      |                   |
| numCl     | Numéro d’un client             | D_NUMCL :<br>NUMBER(6,0)      | Valeurs uniques   |
| nom       | Nom d’un client                | D_NOM :<br>VARCHAR2(25)       |                   |
| prenom    | Prénom d’un client             | D_PRENOM :<br>VARCHAR2(20)    |                   |
| adresse   | Adresse d’un client            | D_ADRESSE :<br>VARCHAR2(40)   |                   |
| cp        | Code postal d’un client        | D_CP : VARCHAR2(5)            |                   |
| ville     | Ville de résidence d’un client | D_VILLE :<br>VARCHAR2(20)     |                   |
| categorie | Catégorie d’un client          | D_CATEGORIE :<br>VARCHAR2(15) | {BON, PRIVILEGIE} |
| nbPers    | Nombre de personnes            | D_NBPERS :<br>NUMBER(2,0)     |                   |
| dateRes   | Date de réservation            | D_DATE : DATE                 |                   |
| code      | Code d’une option              | D_CODE :<br>NUMBER(3,0)       | Valeurs uniques   |
| libelle   | Libellé d’une option           | D_LIBELLE :<br>VARCHAR2(20)   | Valeurs uniques   |

2. Les types syntaxiques, utilisés pour la description des domaines, sont disponibles dans Oracle.

| Attribut | Description       | Domaine                  | Remarques |
|----------|-------------------|--------------------------|-----------|
| prix     | Prix d'une option | D_TARIF :<br>NUMBER(6,2) |           |

#### Remarques

Chaque voyage a une destination composée d'une ville et d'un pays d'arrivée, une ville de départ, le nom de l'hôtel où sont accueillis les clients, son nombre d'étoiles et la durée du voyage en jour.

Pour chaque voyage, plusieurs départs sont planifiés, généralement longtemps à l'avance. Le tarif unitaire, pour une personne, varie selon la période.

Les clients sont identifiés par un numéro et ont différentes caractéristiques dont leur catégorie.

Les clients effectuent des réservations pour des voyages planifiés. Le nombre de personnes et la date de réservation sont conservés pour chaque réservation.

Enfin, des options peuvent être proposées pour les voyages (visite guidée, safari découverte, safari photo, etc.). Ces options peuvent être gratuites pour un voyage ; elles sont alors incluses dans le tarif.

Nous considérons qu'il n'y a pas deux personnes homonymes ayant le même prénom et le même nom.

## 3.2 MCD

Le MCD (voir figure 1) modélise quatre entités :

- **Voyage** : les séjours proposés par le réseau d'agences
- **Client** : les clients du réseau d'agences
- **OptionV** : les options proposées pour les voyages
- **Planning** : les départs planifiés.

L'association identifiante **A** relie un voyage à ses plannings : chaque planning est identifié par sa date de départ au sein d'un voyage donné et porte le tarif unitaire. L'association **Réservation** relie un planning à un client et porte le nombre de personnes ainsi que la date de réservation. Enfin, l'association **Carac** relie un voyage à une option et porte le prix de cette option pour ce voyage.

## 3.3 Schéma relationnel

Les clefs primaires sont soulignées et les clefs étrangères en *italique suivies d'un #*.

Le schéma relationnel de la base de données exemple est présenté ci-après :

Voyage (idV, villeArr, paysArr, villeDep, hotel, nbEtoiles, duree)

Planning (*idV#*, dateDep, tarif)

Client (numCl, nom, prenom, adresse, cp, ville, categorie)

Reservation (*numCl#*, *idV#*, dateDep#, nbPers, dateRes)

OptionV (code, libelle)

Carac (*idV#*, *code#*, prix)

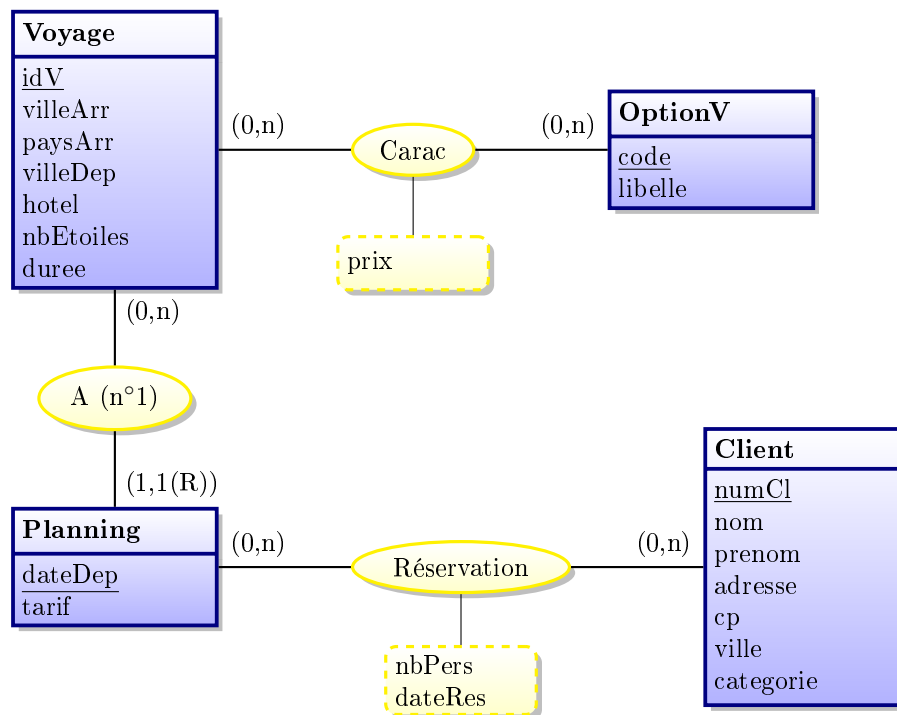


Figure 1 – Modèle conceptuel des données (MCD)

**Remarques**

La clef étrangère composite (idV, dateDep) dans **Reservation** fait référence à la clef primaire composite de **Planning**.

## 4 Requetes avec SQL

Vérifiez que lors du TD précédent, vous avez laissé la base de données dans l'état initial ou réinitialisez-la. Formulez, en SQL, sur la base de données exemple, les requêtes d'interrogation suivantes.

### 4.1 Jointures externes

**Q1** Donnez les numéros et noms des clients ainsi que leurs éventuels identifiants des voyages et dates de réservation datant d'avant le 10/11/2003. *4 attributs, 34 tuples*

```

SELECT C.numCl, nom, idV, dateRes
FROM Client C
    LEFT OUTER JOIN Reservation R
        ON
            C.numCl = R.numCl
            AND dateRes < TO_DATE('2003-11-10', 'YYYY-MM-DD');
  
```

**Q2** En s'inspirant de la requête précédente, donnez les numéros et noms des clients ainsi que leurs éventuels identifiants des voyages et dates de réservation qui n'ont jamais réservé avant le 10/11/2003. *4 attributs, 24 tuples*

```

SELECT C.numCl, nom, idV, dateRes
FROM Client C
  
```

```
LEFT JOIN Reservation R
ON
    C.numCl = R.numCl
    AND dateRes < TO_DATE('2003-11-10', 'YYYY-MM-DD')
WHERE C.numCl IS NULL OR idV IS NULL;
```

**Q3** Formulez les variations de requêtes suivantes de deux manières différentes :

**Q3a** Donnez les numéros et ville de résidence des clients ainsi que les identifiants, villes de départ et pays d'arrivée d'éventuels voyages partant de leur ville de résidence. *5 attributs, 215 tuples*

**V1.**

```
SELECT numCl, ville, idV, villeDep, paysArr
FROM Client
LEFT OUTER JOIN Voyage ON ville = villeDep;
```

**V2.**

```
SELECT numCl, ville, idV, villeDep, paysArr
FROM Voyage
RIGHT OUTER JOIN Client ON ville = villeDep;
```

**Q3b** Donnez les identifiants, villes de départ et pays d'arrivée des voyages ainsi que les éventuels numéros et ville de résidence des clients résidant dans les villes de départ des voyages. *5 attributs, 215 tuples*

**V1.**

```
SELECT idV, villeDep, paysArr, numCl, ville
FROM Client
RIGHT OUTER JOIN Voyage ON ville = villeDep;
```

**V2.**

```
SELECT idV, villeDep, paysArr, numCl, ville
FROM Voyage
LEFT OUTER JOIN Client ON villeDep = ville;
```

**Q4** Formulez les variations de requêtes suivantes à la manière de votre choix :

**Q4a** Donnez les numéros et ville de résidence des clients ainsi que les identifiants, villes de départ et pays d'arrivée d'éventuels voyages à destination du Kenya partant de la ville de résidence des clients. *5 attributs, 46 tuples*

```
SELECT numCl, ville, idV, villeDep, paysArr
FROM Client
LEFT OUTER JOIN Voyage
ON
    ville = villeDep
    AND paysArr = 'KENYA';
```

**Q4b** Donnez les identifiants, villes de départ et pays d'arrivée des voyages à destination du Kenya ainsi que les éventuels numéros et ville de résidence des clients résidant dans les villes de départ des voyages. *5 attributs, 39 tuples*

```
SELECT idV, villeDep, paysArr, numCl, ville
FROM Voyage
  LEFT OUTER JOIN Client ON villeDep = ville
WHERE paysArr = 'KENYA';
```

**Q4c** Donnez les numéros et ville de résidence des clients de numéro supérieur à 2200 ainsi que les identifiants, villes de départ et pays d'arrivée d'éventuels voyages partant de la ville de résidence des clients. *5 attributs, 30 tuples*

```
SELECT numCl, ville, idV, villeDep, paysArr
FROM Client
  LEFT OUTER JOIN Voyage ON ville = villeDep
WHERE numCl > 2200;
```

**Q4d** Donnez les identifiants, villes de départ et pays d'arrivée des voyages ainsi que les éventuels numéros et ville de résidence des clients de numéro supérieur à 2200 résidant dans les villes de départ des voyages. *5 attributs, 41 tuples*

```
SELECT idV, villeDep, paysArr, numCl, ville
FROM Voyage
  LEFT OUTER JOIN Client
    ON
      villeDep = ville
      AND numCl > 2200;
```

**Q5** Donnez les numéros et ville de résidence des clients ainsi que les identifiants, villes de départ et pays d'arrivée d'éventuels voyages de clients habitant une ville d'où ne part aucun voyage pour le Kenya. *5 attributs, 8 tuples*

```
SELECT C.numCl, ville, idV, villeDep, paysArr
FROM Client C
  LEFT OUTER JOIN Voyage
    ON
      ville = villeDep
      AND paysArr = 'KENYA'
WHERE idV IS NULL;
```

**Q6** Donnez les identifiants, villes de départ et pays d'arrivée des voyages ainsi que les éventuels numéros et ville de résidence des clients pour les voyages ne comportant aucun client de numéro supérieur à 2200 résidant dans les villes de départ. *5 attributs, 14 tuples*

```
SELECT idV, villeDep, paysArr, C.numCl, ville
FROM Voyage
  LEFT OUTER JOIN Client C
    ON
      villeDep = ville
      AND numCl > 2200
WHERE numCl IS NULL;
```

## 4.2 Opérations de partitionnement

**Q7** Donnez, par pays d'arrivée, le nombre de voyages. *2 attributs, 7 tuples*

```
SELECT paysArr, COUNT(*) AS nbVoyages
FROM Voyage
GROUP BY paysArr;
```

**Q8** Donnez, par pays et ville d'arrivée, le nombre de voyages. *3 attributs, 15 tuples*

```
SELECT paysArr, villeArr, COUNT(*) AS nbVoyages
FROM Voyage
GROUP BY paysArr, villeArr;
```

**Q9** Donnez, pour chaque pays d'arrivée, le nombre de villes. *2 attributs, 7 tuples*

```
SELECT paysArr, COUNT(DISTINCT villeArr) AS nbVilles
FROM Voyage
GROUP BY paysArr;
```

**Q10** Donnez, pour chaque identifiant et ville d'arrivée de voyage, le nombre de dates planifiées. *3 attributs, 15 tuples*

```
SELECT V.idV, villeArr, COUNT(dateDep) AS nbDates
FROM Voyage V
      INNER JOIN Planning P ON V.idV = P.idV
GROUP BY V.idV, villeArr;
```

**Q11** Donnez, pour chaque identifiant et ville d'arrivée de voyage, le nombre d'options gratuites. *3 attributs, 11 tuples*

```
SELECT V.idV, villeArr, COUNT(code) AS nbOptions
FROM Voyage V
      INNER JOIN Carac Ca ON V.idV = Ca.idV
WHERE prix = 0 OR prix IS NULL
GROUP BY V.idV, villeArr;
```

**Q12** Donnez, par catégorie effective, le nombre de clients. *2 attributs, 2 tuples*

```
SELECT categorie, COUNT(*) AS nbClients
FROM Client
WHERE categorie IS NOT NULL
GROUP BY categorie;
```

**Q13** Donnez, par identifiant de voyage et ville d'arrivée, le nombre de voyages réservés et le nombre total de personnes. *4 attributs, 9 tuples*

```
SELECT V.idV, villeArr, COUNT(R.idV) AS nbReservations, SUM(COALESCE(nbPers, 0)) AS totalPer
FROM Voyage V
      INNER JOIN Reservation R ON V.idV = R.idV
GROUP BY V.idV, villeArr;
```

**Q14** Donnez, par voyage, le prix moyen effectif des options. *2 attributs, 7 tuples*

**V1.**

```
SELECT idV, AVG(prix) AS prixMoyen
FROM Carac
WHERE (prix <> 0 OR prix IS NOT NULL)
GROUP BY idV;
```

**V2.**

```
SELECT idV, AVG(prix) AS prixMoyen
FROM Carac
GROUP BY idV
HAVING AVG(prix) IS NOT NULL;
```

**Q15** Donnez, pour chaque identifiant de voyage et ville d'arrivée, le nombre d'options gratuites et le nombre d'options payantes. *4 attributs, 6 tuples*

```
SELECT V.idV, villeArr, COUNT(DISTINCT CaG.code) AS nbGratuites, COUNT(DISTINCT CaP.code) AS nbPayantes
FROM Voyage V
    INNER JOIN Carac CaG ON V.idV = CaG.idV
    INNER JOIN Carac CaP ON V.idV = CaP.idV
WHERE
    (CaG.prix = 0 OR CaG.prix IS NULL)
    AND (CaP.prix <> 0 OR CaP.prix IS NOT NULL)
GROUP BY V.idV, villeArr;
```

**Q16** Donnez, par ville de résidence, et lorsque son nombre est supérieur à cinq, le nombre de clients. *2 attributs, 2 tuples*

```
SELECT ville, COUNT(*) AS nbClients
FROM Client
GROUP BY ville
HAVING COUNT(*) > 5;
```

**Q17** Quel est, pour chaque identifiant de voyage et pays d'arrivée, le montant total, tenant compte du nombre de personnes, réglé par les clients? *3 attributs, 9 tuples*

```
SELECT V.idV, paysArr, SUM(tarif * nbPers) AS montantTotal
FROM Voyage V
    INNER JOIN Reservation R ON V.idV = R.idV
    INNER JOIN Planning P ON R.idV = P.idV
GROUP BY V.idV, paysArr;
```

**Q18** Quel est, pour chaque identifiant de voyage et date de départ planifiée, le montant total, tenant compte du nombre de personnes, réglé par les clients? *3 attributs, 17 tuples*

```
SELECT P.idV, P.dateDep, SUM(tarif * nbPers) AS montantTotal
FROM Planning P
    INNER JOIN Reservation R
```



```
ON
    P.idV = R.idV
    AND P.dateDep = R.dateDep
GROUP BY P.idV, P.dateDep;
```

**Q19** Quels sont les pays pour lesquels il y a plus de réservations que pour l’Espagne ? *1 attribut, 6 tuples*

```
WITH
    T (paysArr) AS (
        SELECT paysArr
        FROM Voyage V
        INNER JOIN Reservation R ON V.idV = R.idV
    )
SELECT paysArr
FROM T
GROUP BY paysArr
HAVING COUNT(*) > (
    SELECT COUNT(*)
    FROM T
    WHERE paysArr = 'ESPAGNE'
);
```

**Q20** Quelles sont les catégories effectives comportant le plus de clients ? *1 attribut, 1 tuple (PRIVILEGIE)*

```
WITH
    T (categorie, nbClients) AS (
        SELECT categorie, COUNT(*)
        FROM Client
        WHERE categorie IS NOT NULL
        GROUP BY categorie
    )
SELECT categorie
FROM T
WHERE nbClients >= ALL(
    SELECT nbClients
    FROM T
);
```

**Q21** Quels sont les pays pour lesquels il y a le plus de réservations ? *1 attribut, 2 tuples*

```
WITH
    T (paysArr, nbRes) AS (
        SELECT paysArr, COUNT(*)
        FROM Voyage V
        INNER JOIN Reservation R ON V.idV = R.idV
        GROUP BY paysArr
    )
SELECT paysArr
FROM T
WHERE nbRes >= ALL(
```

```
SELECT nbRes
FROM T
);
```

**Q22** Donnez, pour chaque numéro et nom de client ayant fait une réservation pour le Kenya, le nombre total de réservations. *3 attributs, 6 tuples*

```
SELECT C.numCl, nom, COUNT(*) AS nbReservations
FROM Client C
  INNER JOIN Reservation R ON C.numCl = R.numCl
WHERE
  R.numCl IN (
    SELECT numCl
    FROM Reservation R
      INNER JOIN Voyage V ON R.idV = V.idV
    WHERE paysArr = 'KENYA'
  )
GROUP BY C.numCl, nom;
```

**Q23** Donnez, pour chaque pays ayant au moins un hôtel classé cinq étoiles, le nombre total d'hôtels, quel que soit leur nombre d'étoiles. *2 attributs, 3 tuples*

```
SELECT paysArr, COUNT(DISTINCT Hotel) AS nbHotels
FROM Voyage
WHERE
  paysArr IN (
    SELECT paysArr
    FROM Voyage
    WHERE nbEtoiles = 5
  )
GROUP BY paysArr;
```

### 4.3 Requêtes complexes

**Q24** Quelles sont les libellés des options gratuites incluses dans le voyage réservé par le Client Hubert Marin à destination d'Istanbul en date du 04/05/2004 ? *1 attribut, 3 tuples*

```
SELECT DISTINCT libelle
FROM Client C
  INNER JOIN Reservation R ON C.numCl = R.numCl
  INNER JOIN Voyage V ON R.idV = V.idV
  INNER JOIN Carac Ca ON V.idV = Ca.idV
  INNER JOIN OptionV O ON Ca.code = O.code
WHERE
  nom = 'MARIN'
  AND prenom = 'HUBERT'
  AND villeArr = 'ISTANBUL'
  AND dateDep = TO_DATE('2004-05-04', 'YYYY-MM-DD')
  AND (prix = 0 OR prix IS NULL);
```

**Q25** Quelles sont les libellés des options gratuites pour au moins un voyage et payante pour au moins un autre ? Donner deux formulations différentes. *1 attribut, 2 tuples*

V1.

```
SELECT DISTINCT libelle
FROM Carac C1
    INNER JOIN Carac C2 ON C1.code = C2.code
    INNER JOIN OptionV O ON C2.code = O.code
WHERE
    (C2.prix = 0 OR C2.prix IS NULL)
    AND (C1.prix <> 0 OR C1.prix IS NOT NULL);
```

V2.

```
WITH
    T (code, libelle, prix) AS (
        SELECT C.code, libelle, prix
        FROM Carac C
            INNER JOIN OptionV O ON C.code = O.code
    )
SELECT DISTINCT libelle
FROM T
WHERE
    (prix = 0 OR prix IS NULL)
    AND code IN (
        SELECT code
        FROM T
        WHERE (prix <> 0 OR prix IS NOT NULL)
    );
```

**Q26** Quelles sont les libellés des options toujours gratuites? Proposer quatre formulations différentes. *1 attribut, 5 tuples*

V1.

```
SELECT libelle
FROM OptionV
WHERE code NOT IN (
    SELECT code
    FROM Carac
    WHERE (prix <> 0 OR prix IS NOT NULL)
);
```

V2.

```
SELECT libelle
FROM OptionV
WHERE code <> ALL(
    SELECT code
    FROM Carac
    WHERE (prix <> 0 OR prix IS NOT NULL)
);
```

V3.

```
SELECT libelle
FROM OptionV O
WHERE NOT EXISTS (
    SELECT *
    FROM Carac Ca
    WHERE
        Ca.code = O.code -- noqa: RF03
        AND (prix <> 0 OR prix IS NOT NULL) -- noqa: RF03
);
```

V4.

```
SELECT DISTINCT libelle
FROM OptionV O
    LEFT OUTER JOIN Carac Ca
        ON
            O.code = Ca.code
            AND (prix <> 0 OR prix IS NOT NULL)
WHERE idV IS NULL;
```

V5.

```
SELECT libelle
FROM OptionV
WHERE code IN (
    SELECT code
    FROM Carac
    EXCEPT
    SELECT code
    FROM Carac
    WHERE (prix <> 0 OR prix IS NOT NULL)
);
```

**Q27** Quels sont les numéros, noms et prénoms des autres clients ayant réservé pour un des voyages réservé par Arnaud Peyroche? *3 attributs, 7 tuples*

```
WITH
    T (numCl, nom, prenom, idV) AS (
        SELECT C.numCl, nom, prenom, idV
        FROM Client C
            INNER JOIN Reservation R ON C.numCl = R.numCl
    )
SELECT DISTINCT numCl, nom, prenom
FROM T
WHERE
    (nom <> 'PEYROCHE' OR prenom <> 'ARNAUD')
    AND idV IN (
        SELECT idV
        FROM T
```

```
WHERE
    nom = 'PEYROCHE'
    AND prenom = 'ARNAUD'
);
```

**Q28** Donnez le nombre de réservations et le nombre total de personnes pour les clients de la catégorie privilégié. *2 attributs, 1 tuple (6, 10)*

```
SELECT COUNT(*) AS nbReservations, SUM(COALESCE(nbPers, 0)) AS totalPers
FROM Client C
    INNER JOIN Reservation R ON C.numCl = R.numCl
WHERE categorie = 'PRIVILEGIE';
```

| Remarques                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Il faut gérer les valeurs nulles dans le calcul horizontal, car si nbPers prend une valeur nulle, sa somme sera à NULL et la fonction agrégative SUM rendra un résultat faux. |

**Q29** Quels sont les numéros, noms et prénoms des clients ayant fait une réservation avec un nombre total maximal de personnes ? *3 attributs, 1 tuple*

```
WITH
    T (numCl, nom, prenom, nbPers) AS (
        SELECT C.numCl, nom, prenom, nbPers
        FROM Client C
            INNER JOIN Reservation R ON C.numCl = R.numCl
    )
SELECT numCl, nom, prenom
FROM T
WHERE COALESCE(nbPers, 0) = (
    SELECT MAX(COALESCE(nbPers, 0))
    FROM T
);
```

**Q30** Quel est le montant total des réservations du Client numéro 2101 ? *1 attribut, 1 tuple (14484)*

```
SELECT SUM(COALESCE(nbPers, 0) * COALESCE(tarif, 0))
FROM Reservation R
    INNER JOIN Planning P ON R.idV = P.idV
WHERE numCl = 2101;
```

**Q31** Quels sont les hôtels avec le plus d'étoiles ? *1 attribut, 4 tuples*

```
SELECT DISTINCT Hotel
FROM Voyage
WHERE nbEtoiles = (
    SELECT MAX(nbEtoiles)
    FROM Voyage
);
```

**Q32** Quels sont les pays n'ayant pas d'hôtel avec le plus grand nombre d'étoiles ? *1 attribut, 4 tuples*

```
SELECT DISTINCT paysArr
FROM Voyage
WHERE paysArr NOT IN (
    SELECT paysArr
    FROM Voyage
    WHERE nbEtoiles = (
        SELECT MAX(nbEtoiles)
        FROM Voyage
    )
);
```

**Q33** Donnez les numéros et ville de résidence des clients, y compris ceux sans voyage, ainsi que les identifiants, villes de départ et pays d'arrivée des voyages, y compris ceux sans client, à destination du Kenya partant de la ville de résidence des clients. *5 attributs, 66 tuples*

```
SELECT numCl, ville, idV, villeDep, paysArr
FROM Client
    FULL OUTER JOIN Voyage
    ON
        ville = villeDep
        AND paysArr = 'KENYA';
```

**Q34** Quels sont les numéros et noms des clients qui n'ont fait aucune réservation pour un voyage au Maroc? Donnez deux formulations : une avec jointure externe et une avec test de non-existence. *2 attributs, 26 tuples*

Version avec jointure externe.

```
SELECT C.numCl, nom
FROM Client C
    LEFT OUTER JOIN (
        SELECT *
        FROM Voyage V
            INNER JOIN Reservation R ON V.idV = R.idV
        WHERE paysArr = 'MAROC'
    ) T
    ON C.numCl = T.numCl
WHERE T.numCl IS NULL;
```

Version avec test de non existence.

```
SELECT C.numCl, C.nom
FROM Client C
WHERE NOT EXISTS (
    SELECT *
    FROM Voyage V
        INNER JOIN Reservation R ON V.idV = R.idV
    WHERE
        paysArr = 'MAROC'
        AND R.numCl = C.numCl
);
```